

Automatic Cat Feeder – Full Summary

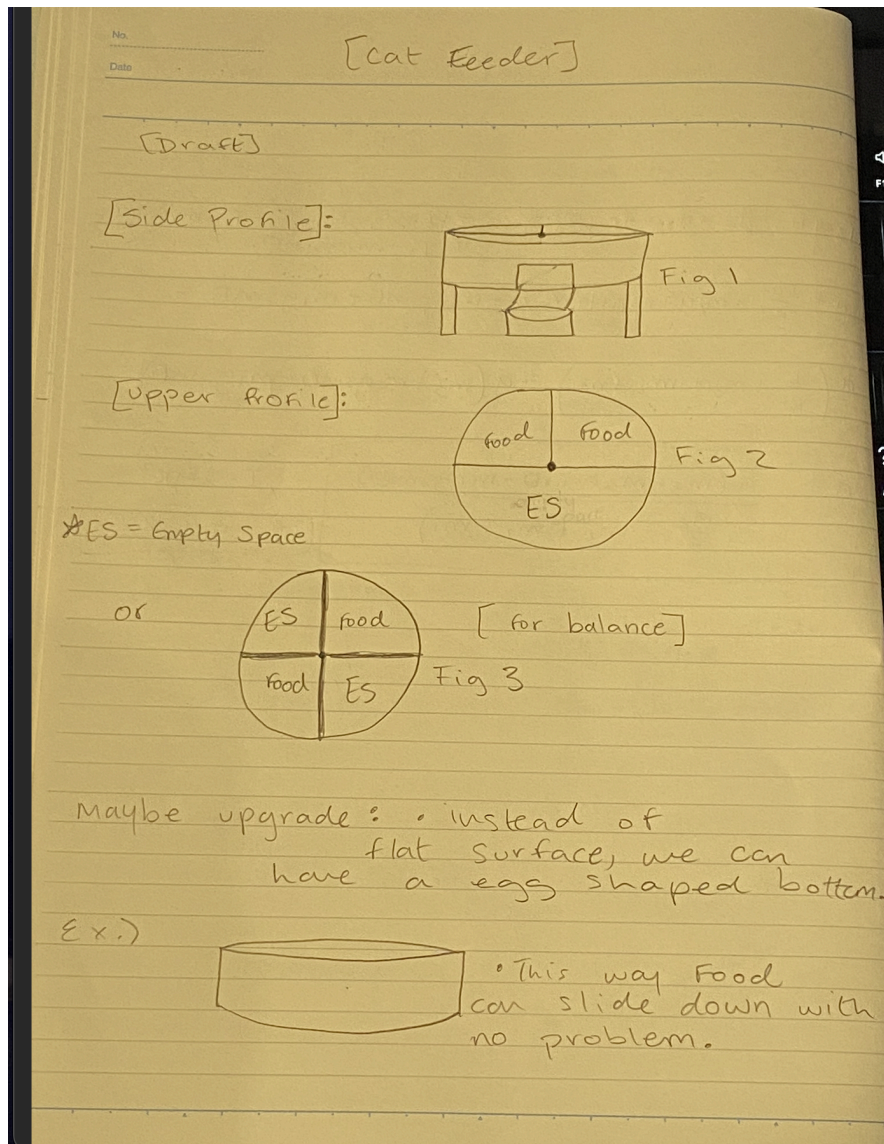
Project Concept

- The feeder is a **Ferris wheel-style rotating dispenser** for cat food.
 - The wheel is laying flat on the ground, not vertically.
 - **Two feedings per day**: 9 AM and 6 hours later at 3 PM
 - A **motor rotates the wheel**, which releases food into a bowl.
 - The **RTC module (DS3231)** is used for **timekeeping and alarms**
 - The plan is to have no computer/Arduino in final design.
 - A **transistor** flips the active-low alarm signal to control the motor.
 - **Diode protects the transistor from back-EMF** from the motor.
-

Feeder Physical Design (Cardboard / Prototype / 3D print)

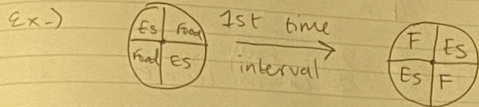
- **Wheel dimensions**:
 - Diameter: 12 in (6 in radius)
 - Height (thickness of wheel): 2 in
- **Standing legs**: 6 in tall
- **Bottom enclosure for electronics**: 2 in tall × 6 in length
- **Slide for dispensing food**: 2–3 in long
- **Wheel sections**: 3 sections total (2 for food, 2 empty for spacing)

Picture of my notebook draft: btw the pics for some reason turned out super yellow and ugly so I'm sorry about that.

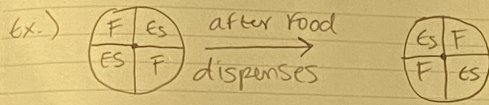


[DC motor]:

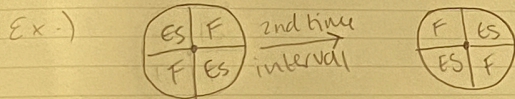
- If it starts like Fig 3, will need to rotate 45° for food to dispense.



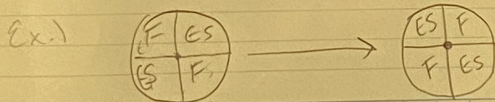
- Food empties after 1st time interval, but right after the dc motor moves another 45° so it'll be in ES.



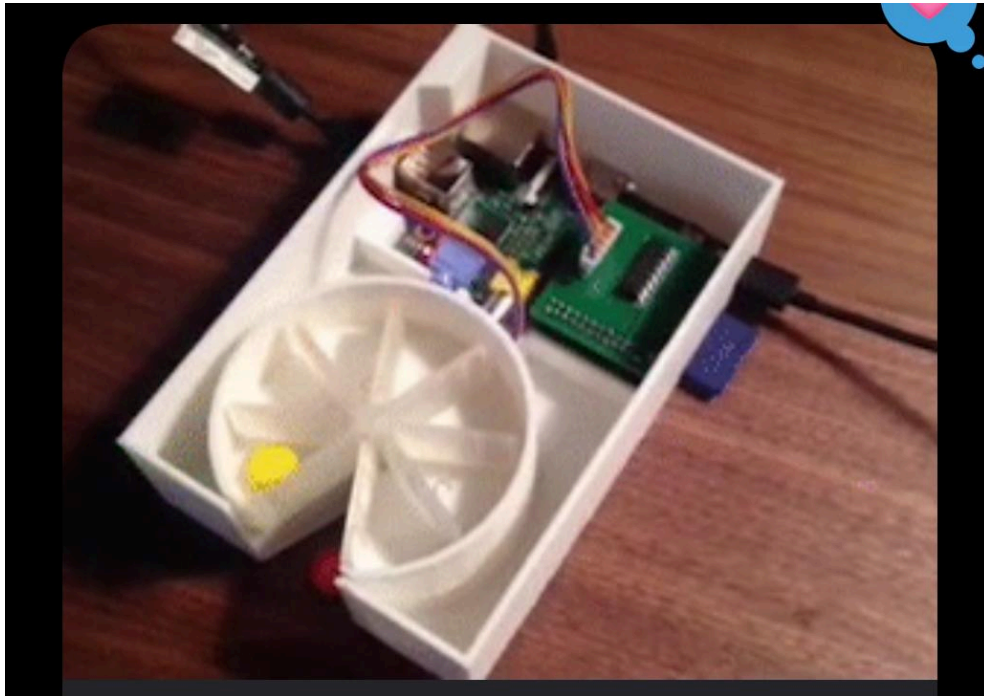
- Dc motor spins 45° when 2nd time interval occurs. Food dispenses



- Dc motor rotates back to its original state until next day when time intervals occur again.



Picture that Ethan sent to me that inspired the design:



Components

1. Motor:

- DC geared motor, 3 RPM, 9 V (slow enough to dispense food carefully)
- Diameter: 40 mm
- Shaft: 4–5 mm
- Needs wires soldered to motor terminals

2. Battery:

- 1 × 9 V battery with holder and ON/OFF switch
- Leads: Red (+), Black (–)

3. RTC Module:

- DS3231 AT24C32
- Power: 3.3–5.5 V

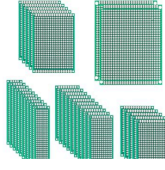
- I²C interface (SCL, SDA)
- Alarm outputs (active low) for triggering motor
- 4. **Transistor:**
 - NPN (e.g., 2N2222) to switch motor on/off with RTC output
 - Resistor (~1 k Ω) between RTC alarm output and transistor base
- 5. **Diode (flyback diode):**
 - Across motor terminals to protect transistor from back-EMF
 - Cathode to motor +, Anode to motor –
- 6. **Other materials:**
 - Perfboard or if we don't use perfboard, breadboard will work.
 - Wires
 - Solder
 - Small enclosure or box for electronics

Supplies I already have:

- NPN transistor
- Diode
- Resistors
- wires

Supplies Needed:

- RTC module
- DC motor
- Battery pack
- Perfboard
- Pictures of supplies I will be buying:

	LampVPath (Pack of 2 9v Battery Holder, 9 Volt Battery Holder with Switch, 9v Battery Case with Switch) \$5⁹⁹ (\$3.00 / count) In Stock FREE delivery Thu, Apr 9 available at checkout FREE Returns ☐ This is a gift Learn more 1 + Delete Save for later Share
	Fayelume 12 V DC Motor, 3 RPM Slow Rotating Motors, Small DC Motor, 4 MM Shaft Diameter Motors, for Small Display Stand, Jewelry Display \$8⁹⁹ Only 8 left in stock - order soon. FREE delivery Thu, Apr 9 available at checkout FREE Returns ☐ This is a gift Learn more 1 + Delete Save for later Share
KOKUYO Spiral Notebook with 5x5 Graph Paper, 3 Pack Campus 10-1/2"... was removed from Shopping Cart.	
	HiLetgo 5pcs DS3231 AT24C32 Clock Module Real Time Clock Module IIC RTC Module for Arduino Without Battery \$14⁹⁹ #1 Best Seller in Clock Integrated Circuits In Stock FREE delivery Thu, Apr 9 available at checkout FREE Returns ☐ This is a gift Learn more Size: Small 1 + Delete Save for later Share
	Rindion 32 Pcs PCB Board, Green Circuit Board with 5 Sizes Compatible, Double Sided PCB Prototype Board for DIY Electronics Projects Apply to Soldering Projects \$7⁹⁹ In Stock FREE delivery Thu, Apr 9 available at checkout FREE Returns ☐ This is a gift Learn more 1 + Delete Save for later Compare with similar items Share

Real-Life Wiring

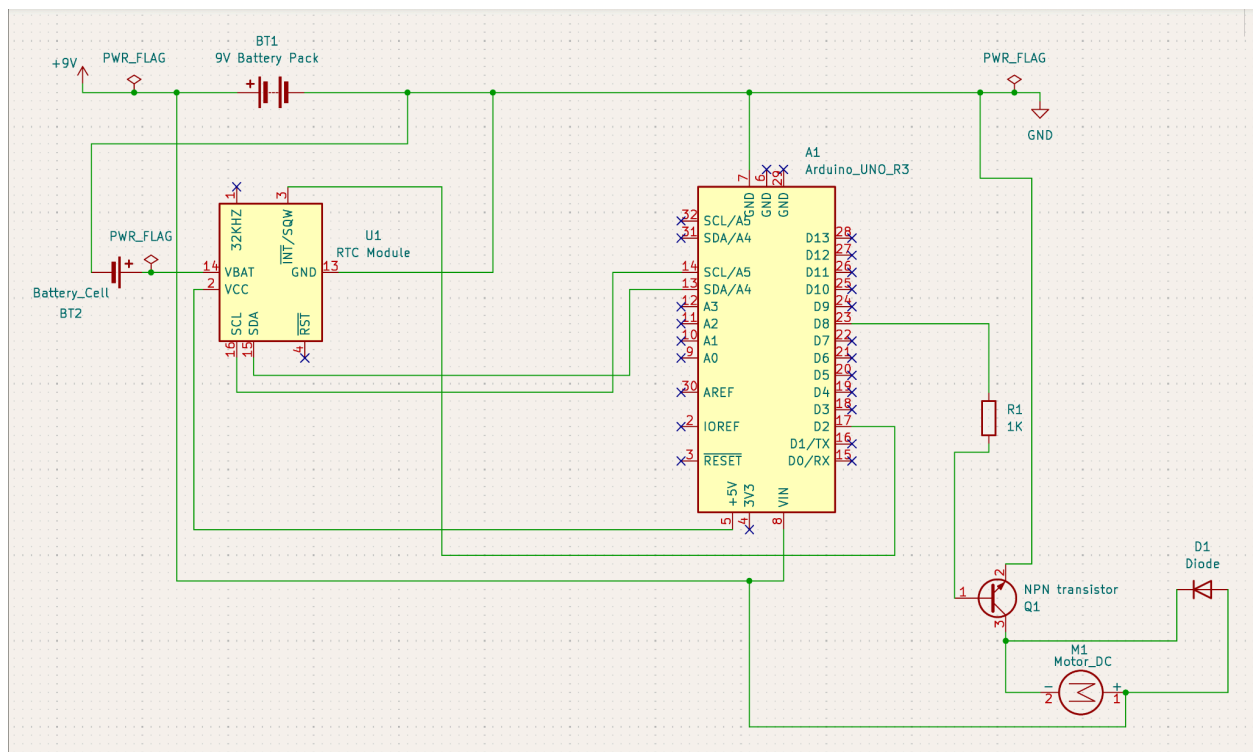
- Battery → Motor:**
 - Battery + → Motor +
 - Motor – → Collector of NPN transistor
 - Emitter → Battery –
- Diode:**
 - Cathode → Motor +
 - Anode → Motor –
- RTC Alarm → Transistor Base:**
 - RTC alarm output → 1 kΩ resistor → Base of transistor
- RTC GND → Battery –**
- Optional manual button:**
 - Between base of transistor and battery + (or GND depending on logic)
- Power for RTC:**

- VCC → Battery + (or regulated 5 V if using a separate voltage source)
- GND → Battery –

Notes:

- The diode prevents voltage spikes from the motor damaging the transistor.
- The battery powers everything; the RTC provides the “logic” without Arduino.

Schematic



Feeder Operation

1. RTC module is programmed with two alarm times (e.g., 9 AM, 3 PM).
2. When alarm triggers:
 - RTC output goes low → transistor turns motor ON → wheel rotates → food dispenses.
3. Motor stops after set rotation (~1–2 rotations).
4. Diode protects electronics from motor back-EMF.

Questions/Notes

Questions:

-

Notes:

- There will be a lot of things that will be un-perfect, but it doesn't have to be perfect since we only have like 2-3 weeks left before we present. So i was thinking as long as it runs we go with that and the stuff that needs fixing, we can write in our report as future improvements.

Concerns:

- **First concern:** How long should we keep the motor running after the RTC alarm goes off?
 - Slight context:

- There will be a permanent opening where the bottom right ES is and once the rtc hits 9am, the wheel spins 45 degrees (idk how long, but maybe 5-10 sec) and dispense the next section aka top right. If the motor keeps moving past the food section, the ES will be the next section, so no food dispenses and it will surely stop by then. The RTC has the other interval until 3pm so when it hits 3pm, the alarm moves the motor again and the food dispenses with the next food section. This time there is no more food so even if the motor keeps moving, there won't be overfeeding.
- I was thinking after we buy the dc motor, we test the speed of it and estimate how long we can keep it going for.
- **Second concern:** How will we build our cat feeder wheel?
 - 3D printing the case seems like the best option for our project, but I've never designed a 3D print before. I've attempted but failed miserably.
 - If any of you two know how to or want to attempt to build it, please do!

Code

```
#include <Wire.h>

#include "RTClib.h"

RTC_DS3231 rtc;

int motorPin = 8;

int motorRunTime = 8000;

int feedHour = 19;

int feedMinute = 6;

bool alreadyFed = false;

unsigned long lastCheck = 0;

void setup() {

    Serial.begin(9600);

    pinMode(motorPin, OUTPUT);

    digitalWrite(motorPin, LOW);

    if (!rtc.begin()) {

        Serial.println("RTC not found");

        while (1);

    }

    // Only sets time if RTC lost power/battery

    if (rtc.lostPower()) {
```

```
Serial.println("RTC lost power, setting time...");

rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

}

Serial.println("System Ready");

}

void loop() {

    if (millis() - lastCheck >= 1000) {

        lastCheck = millis();

        DateTime now = rtc.now();

        int hour = now.hour();

        int minute = now.minute();

        int second = now.second();

        Serial.print("TIME: ");

        Serial.print(hour);

        Serial.print(":");

        Serial.print(minute);

        Serial.print(":");

        Serial.println(second);

        if (hour == feedHour && minute == feedMinute && !alreadyFed) {

            Serial.println("FEEDING STARTED");
```

```

    digitalWrite(motorPin, HIGH);

    delay(motorRunTime);

    digitalWrite(motorPin, LOW);

    alreadyFed = true;

}

if (!(hour == feedHour && minute == feedMinute)) {

    alreadyFed = false;

}

}

}

```

Maybe diff code if needed but need to test bc not fully sure if right - Jacob

```

#include <Wire.h>

#include "RTClib.h"

RTC_DS3231 rtc;

int motorPin = 8;

unsigned long motorRunTime = 8000;

// Feeding schedule (24-hour time)
int feedTimes[][2] = {

    {9, 0}, // 9:00 AM

    {15, 0} // 3:00 PM

```

```
};
```

```
const int numFeedings = 2;
```

```
bool alreadyFed[numFeedings] = {false, false};
```

```
// Motor state (non-blocking)
```

```
bool motorRunning = false;
```

```
unsigned long motorStartTime = 0;
```

```
// Timing for checking clock
```

```
unsigned long lastCheck = 0;
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  pinMode(motorPin, OUTPUT);
```

```
  digitalWrite(motorPin, LOW);
```

```
  if (!rtc.begin()) {
```

```
    Serial.println("RTC not found");
```

```
    while (1);
```

```
  }
```

```
  if (rtc.lostPower()) {
```

```
    Serial.println("RTC lost power, setting time...");
```

```
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
```

```
  }
```

```
  Serial.println("System Ready");
```

```
}
```

```
void loop() {
```

```
  // Check time every second
```

```

if (millis() - lastCheck >= 1000) {

    lastCheck = millis();

    DateTime now = rtc.now();

    int hour = now.hour();

    int minute = now.minute();

    int second = now.second();

    // Print time with leading zeros

    Serial.print("TIME: ");

    if (hour < 10) Serial.print("0");

    Serial.print(hour);

    Serial.print(":");

    if (minute < 10) Serial.print("0");

    Serial.print(minute);

    Serial.print(":");

    if (second < 10) Serial.print("0");

    Serial.println(second);

    // Check each feeding time

    for (int i = 0; i < numFeedings; i++) {

        if (hour == feedTimes[i][0] &&

            minute == feedTimes[i][1] &&

            second < 10 &&

            !alreadyFed[i] &&

            !motorRunning) {

            Serial.print("FEEDING #");

            Serial.println(i + 1);

            digitalWrite(motorPin, HIGH);

            motorStartTime = millis();

```



```
    motorRunning = true;

    alreadyFed[i] = true;

}

// Reset after minute passes

if (!(hour == feedTimes[i][0] && minute == feedTimes[i][1])) {

    alreadyFed[i] = false;

}

}

}

// Stop motor after run time (non-blocking)

if (motorRunning && millis() - motorStartTime >= motorRunTime) {

    digitalWrite(motorPin, LOW);

    motorRunning = false;

    Serial.println("FEEDING COMPLETE");

}

}
```